

GramWork AI – Smart Village Labor Management System

I want you to act as a senior full-stack software architect, UI/UX designer, database engineer, and AI product developer.

Build a complete production-ready web platform called:

“GramWork AI – Smart Village Labor Management System”

OBJECTIVE:

Create a modern AI-powered rural employment platform that connects village laborers with nearby work opportunities based on GPS location, skills, availability, urgency, and wage preferences.

The platform should help:

- Laborers find nearby jobs
- Farmers and contractors hire workers easily
- Panchayat/government authorities monitor employment
- Admin manage the entire ecosystem
- Improve transparency, employment, and digital transformation in villages

TECH STACK

Frontend:

- React.js + Vite
- Tailwind CSS
- Framer Motion
- React Router
- Axios
- Redux Toolkit

Backend:

- Node.js + Express.js

Database:

- MongoDB + Mongoose

Authentication:

- JWT Authentication
- OTP Mobile Verification

Maps:

- Leaflet.js + OpenStreetMap

Real-time:

- Socket.io

Deployment:

- Docker
- Vercel (Frontend)

- Render/Railway (Backend)
- MongoDB Atlas

MAIN FEATURES

1. ROLE-BASED SYSTEM

Create 3 main roles:

- Admin
- Employer / Contractor
- Laborer / Worker

Each role should have:

- Separate dashboard
- Separate permissions
- Custom analytics
- Secure routes

2. AUTHENTICATION SYSTEM

Implement:

- Register/Login
- JWT authentication
- Role-based authorization
- OTP verification
- Forgot password
- Reset password
- Session handling
- Google authentication (optional)

3. LABORER PROFILE

Fields:

- Full name
- Age
- Gender
- Phone number
- Aadhaar verification
- Profile image
- Current GPS location
- Village name
- Skills
- Years of experience
- Daily wage expectation
- Availability status
- Working hours
- Preferred work category
- Languages known
- Ratings & reviews

Skills examples:

- Farming
- Construction
- Mason
- Carpenter
- Electrician
- Plumber
- Painter
- Driver
- Cleaner
- Daily wage labor

4. EMPLOYER FEATURES

Employers should be able to:

- Post jobs
- Mention required workers
- Set salary/wage
- Add work duration
- Add urgency level
- Upload work images
- Add exact location
- View nearby workers
- Assign workers
- Mark attendance
- Track work progress
- Release payments
- Rate workers

5. AI-POWERED LABOR ASSIGNMENT

Build an intelligent worker recommendation system based on:

- GPS distance
- Skill match
- Wage compatibility
- Availability
- Ratings
- Previous work history
- Reliability score

Features:

- AI worker ranking
- Auto-suggest workers
- Nearby worker map
- Smart matching algorithm
- Accept/reject assignment
- Priority assignment logic

6. LOCATION & MAP FEATURES

=====

Implement:

- Live GPS tracking
- Interactive map
- Search by village or pincode
- Distance calculation
- Nearby jobs
- Nearby workers
- Route navigation
- Geo-tagged jobs
- Real-time worker location

=====

7. ADMIN PANEL

=====

Admin dashboard should include:

- Total workers
- Total jobs
- Active assignments
- Employment analytics
- Village-wise employment stats
- Worker verification
- Employer verification
- Fraud detection
- Complaint/dispute management
- Report generation

Analytics charts:

- Employment growth
- Daily jobs
- Worker participation
- Wage trends
- Village activity heatmaps

=====

8. REAL-TIME FEATURES

=====

Use Socket.io for:

- Real-time notifications
- Live assignment updates
- Chat system
- Instant job alerts
- Live attendance updates

Notifications:

- In-app
- SMS-ready architecture
- WhatsApp-ready integration
- Email notifications

9. PAYMENT SYSTEM

Implement:

- Wage tracking
- Worker earnings dashboard
- UPI integration-ready structure
- Pending/completed payments
- Salary history
- Invoice generation
- Payment receipts

10. UI/UX REQUIREMENTS

Design should look:

- Modern startup quality
- Professional government-tech platform
- Rural-friendly
- Clean and minimal
- Mobile-first responsive

Use:

- Tailwind CSS
- Glassmorphism effects
- Beautiful gradients
- Soft shadows
- Smooth animations
- Large accessible buttons
- Modern typography

Pages:

- Landing page
- About page
- Contact page
- Job listings
- Worker listings
- Dashboard pages
- Admin analytics
- Profile pages

Landing page sections:

- Hero section
- Features
- Statistics
- How it works
- Testimonials
- Mission statement
- Footer

11. MULTILINGUAL SUPPORT

Support:

- English
- Hindi

Include:

- Language switcher
- Hindi-friendly UI
- Voice search in Hindi

12. ADVANCED FEATURES

Add:

- AI chatbot assistant
- Voice search
- QR attendance system
- Worker check-in/check-out
- Emergency SOS button
- Offline-first PWA support
- Dark/light mode
- Smart filters
- Search suggestions
- AI fraud detection
- AI worker recommendation engine

13. DATABASE DESIGN

Create optimized MongoDB schemas for:

- Users
- Jobs
- Assignments
- Payments
- Attendance
- Reviews
- Notifications
- Villages
- Messages
- Complaints

14. API DEVELOPMENT

Build professional REST APIs using MVC architecture.

Include APIs for:

- Authentication
- Jobs
- Workers
- Assignments

- Payments
- Reviews
- Notifications
- Analytics
- Attendance
- Chat

15. SECURITY

Implement:

- Password hashing
- JWT security
- Rate limiting
- Helmet security
- Input validation
- API sanitization
- Role protection
- File upload validation

16. PERFORMANCE OPTIMIZATION

Add:

- Lazy loading
- Pagination
- Infinite scrolling
- Image optimization
- SEO optimization
- Error boundaries
- Skeleton loaders
- Toast notifications
- Code splitting
- API caching

17. PROJECT STRUCTURE

Generate:

- Clean folder structure
- Reusable components
- Environment setup
- API service layer
- Redux state management
- Docker configuration
- README.md
- Installation guide
- Deployment steps

18. FINAL OUTPUT REQUIRED

=====

Generate:

1. Full frontend code
2. Full backend code
3. MongoDB schemas
4. REST APIs
5. Responsive UI
6. Dashboard system
7. Dummy/sample data
8. README documentation
9. Docker setup
10. Deployment guide
11. .env.example
12. API testing examples

=====

19. EXTRA IMPORTANT REQUIREMENTS

=====

- Follow scalable enterprise-level architecture
- Use clean code practices
- Use reusable components
- Make UI extremely beautiful and modern
- Optimize for low internet speed in rural areas
- Ensure mobile-first design
- Add comments in important code sections
- Use best practices for security and performance

The final result should look like a real-world startup product for rural workforce digitization and smart labor assignment management.

Start by:

1. Designing the complete folder structure
2. Creating backend architecture
3. Creating frontend architecture
4. Designing MongoDB schemas
5. Building authentication system
6. Then continue module by module with complete production-ready code.